

◆ .NET Backend (C# / ASP.NET Core / .NET 6/7/8)

✓ Basics

- What are the differences between **.NET Core** and **.NET Framework**?
- Explain the **Middleware** pipeline in ASP.NET Core.
- What is **Dependency Injection**, and how is it implemented in .NET Core?
- Difference between IEnumerable, IQueryable, and List?
- Explain the concept of **async/await** in C#.

✓ Advanced

- What are **filters** in ASP.NET Core (e.g., Authorization, Resource, Action filters)?
- How do you implement **custom middleware**?
- How does **model binding** work in Web API?
- What are **records** in C# 9+?
- Explain **Entity Framework Core Migrations**.

◆ Frontend (Angular / React)

Angular-Specific (Replace with React as needed):

- What are **components**, **modules**, and **services** in Angular?
- How do you implement **lazy loading**?

- What is **RxJS**, and when do you use **Observables** vs **Promises**?
- What are **lifecycle hooks** in Angular?
- How do you communicate between two unrelated components?

General Frontend

- How do you manage **form validation**?
 - Difference between **template-driven** and **reactive forms**?
 - What is the use of **ngOnInit**?
 - How do you handle **state management**?
-

◆ Database (SQL Server / PostgreSQL / MySQL)

- How do you write optimized SQL queries?
 - Explain **joins**: inner, left, right, full.
 - What are **stored procedures**, and when should you use them?
 - How do you handle **transaction management**?
 - How would you debug a **slow-performing query**?
-

◆ API Development & Integration

- What is the difference between **REST** and **SOAP**?
- How do you **version** a Web API?

- How do you **secure** an API (JWT / OAuth2)?
 - What is **Swagger/OpenAPI**?
 - How do you handle **global exception handling** in Web API?
-

◆ Design Patterns & Architecture

- What is the **Repository Pattern** and **Unit of Work**?
 - What is **SOLID**? Can you explain each principle?
 - What is the difference between **monolithic** and **microservices**?
 - When would you use a **CQRS** pattern?
-

◆ DevOps / CI-CD / Testing (Optional for full-stack)

- What is your experience with **Azure DevOps / GitHub Actions / Jenkins**?
 - Have you worked with **Docker** in a .NET project?
 - How do you **automate builds and deployments**?
 - How do you write **unit tests** for controllers and services?
 - What tools do you use for **code coverage**?
-

◆ Behavioral / Problem Solving

- Tell me about a challenging full-stack bug you fixed.
- How do you manage communication between backend and frontend teams?
- How do you handle performance bottlenecks?
- What tools do you use to debug frontend/backend issues?

Difference between IEnumerable, IQueryable, and List?

Quick Summary

Feature	IEnumerable<T>	IQueryable<T>	List<T>
Interface/Type	Interface	Interface	Concrete class
Namespace	System.Collections.Generic	System.Linq	System.Collections.Generic
Execution	Deferred	Deferred	Immediate
Query Processing	In-memory	At DB/server (for EF)	In-memory
Use with LINQ	LINQ to Objects	LINQ to Entities/SQL	LINQ to Objects
Use case	Looping over collections	Querying from database	When you need a full in-memory collection
Modifiable	✗ (unless casted)	✗	✓
Performance (for DB)	Poor (data already fetched)	High (query translated to SQL)	N/A

◆ IEnumerable<T>

- Designed to **iterate over a collection**.
- Suitable for **in-memory** operations (like arrays, lists).
- Executes LINQ queries in **memory**.

csharp

CopyEdit

```
IEnumerable<User> users = db.Users.ToList(); // DB hit happens here
```

```
var filtered = users.Where(u => u.Age > 30); // Filtering done in memory
```

! This can lead to performance issues with large datasets because the entire list is already in memory.

◆ IQueryable<T>

- Inherits from IEnumerable<T>.
- Designed for **querying external sources**, like **databases**.
- Executes queries **remotely** (e.g., SQL Server).
- Supports **deferred execution**, but query is translated to **SQL** and executed on the server.

csharp

CopyEdit

```
IQueryable<User> users = db.Users; // No DB call yet
```

```
var filtered = users.Where(u => u.Age > 30); // Still no DB call
```

```
var result = filtered.ToList(); // SQL query sent to DB and executed
```

✓ Efficient for large datasets, only the required data is fetched from the DB.

◆ List<T>

- A concrete class that holds items **in memory**.
- Data is **immediately fetched** and fully loaded.
- LINQ operations are done in memory.

csharp

CopyEdit

```
List<User> users = db.Users.ToList(); // DB hit immediately
```

```
var filtered = users.Where(u => u.Age > 30); // In-memory filter
```

✅ Best when you need full data and plan to manipulate or iterate multiple times.

✅ When to Use What?

Use Case	Recommended
Fetching from DB with filters	IQueryable<T>
Working with in-memory collections	IEnumerable<T> or List<T>
Adding/removing elements dynamically	List<T>
Lazy-loaded server-side query	IQueryable<T>
Looping only	IEnumerable<T>

 Bonus Tip:

- Use `IQueryable<T>` **until the final moment**, then use `.ToList()` or `.FirstOrDefault()` to execute the query.